NAME: CYBER SECURITY



Assignment Number 2

Kali Practice

Name: Syed Muhammad Abdullah Shah

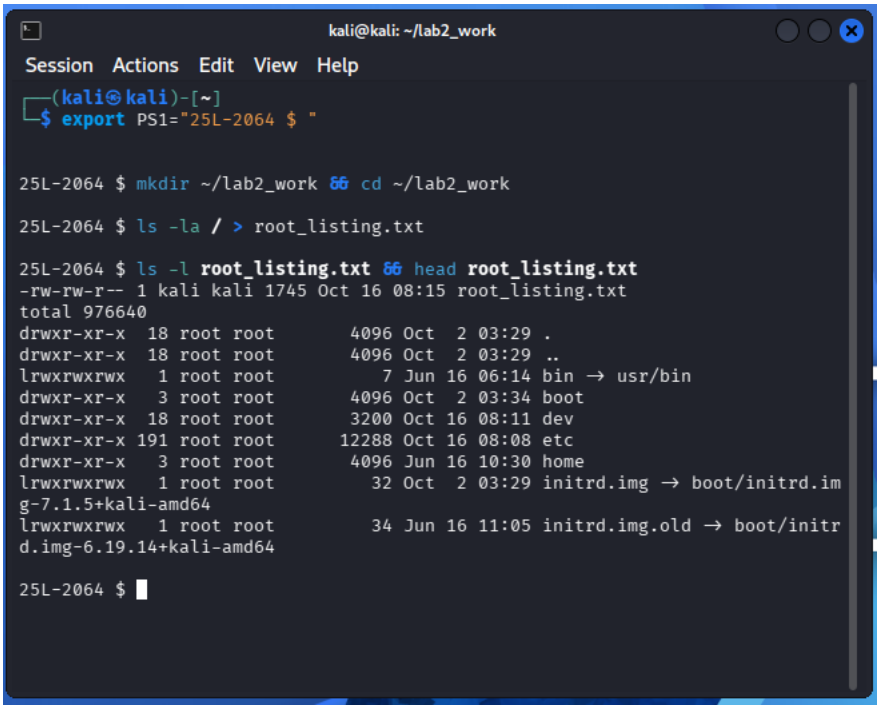
Roll No: 25L-2064

Section: BSCY-3A

Saturday 16 October 2026

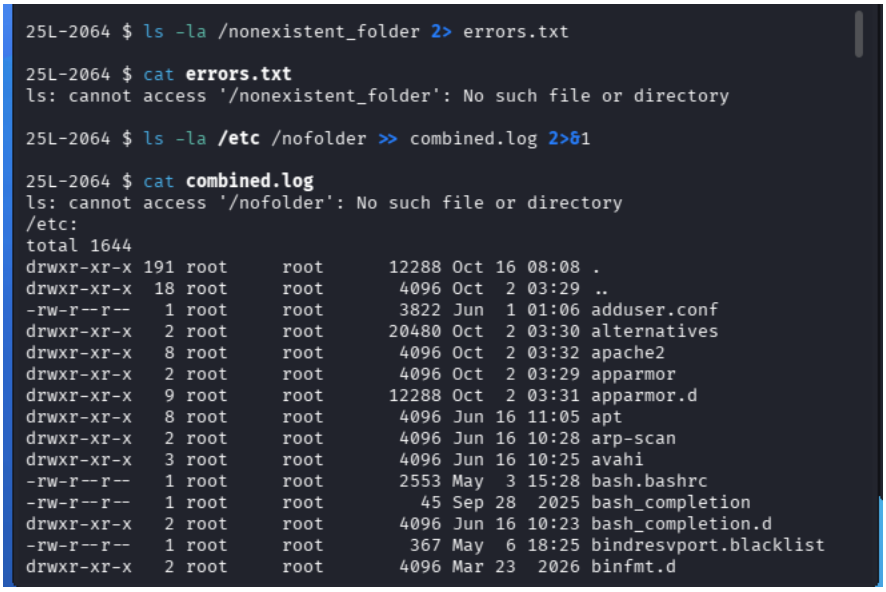
FAST School of Computing, Lahore Campus

National University of Computer and Emerging Sciences



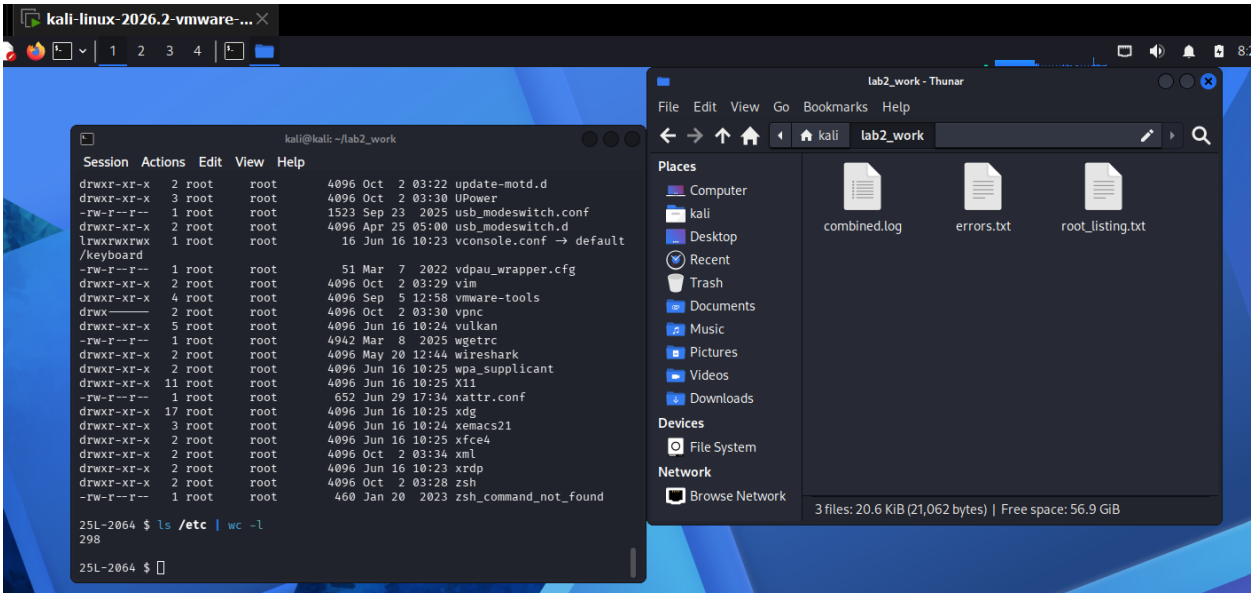
Explanation:

I created a folder called lab2_work and moved into it. Then I used > to save the output of ls -la / into root_listing.txt instead of printing it on screen. I checked the file with ls -l and head, and it was created.



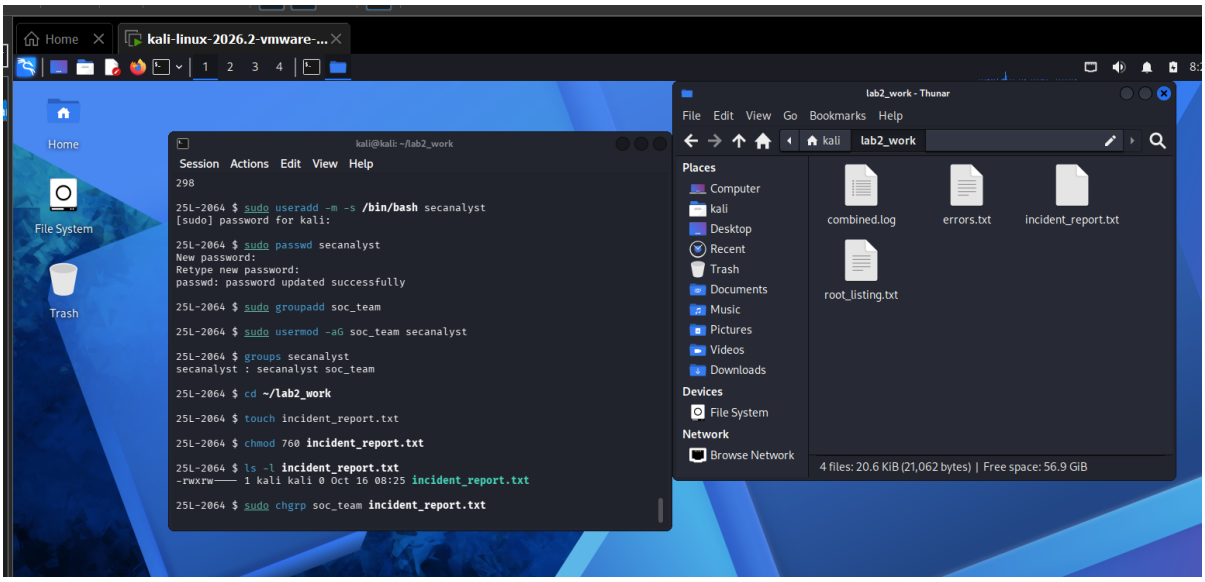
Explanation:

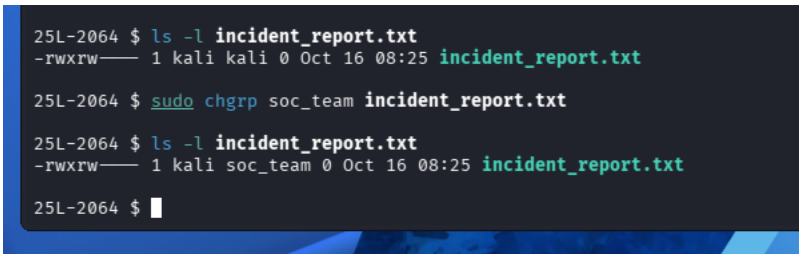
The folder /nonexistent_folder does not exist, so ls gave an error. Using 2> sent that error message into errors.txt instead of the screen. Then I used >> combined.log 2>&1 to add both the normal output of /etc and the error for /nofolder to one file. >> adds to the file without deleting what is already inside.



Explanation:

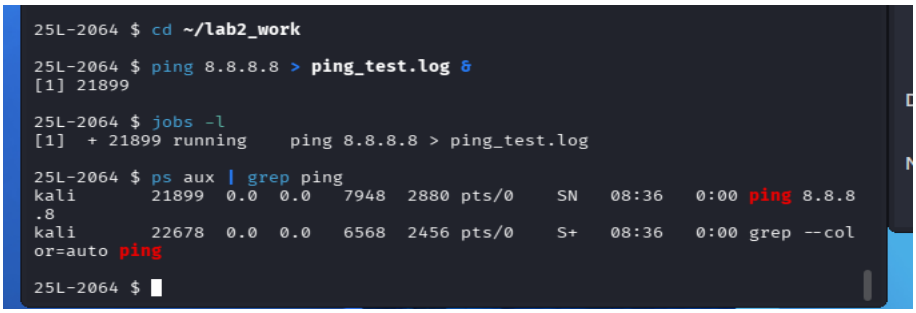
I used ls /etc | wc -l. The pipe | sends the list of files from ls into wc -l, which counts the lines. The result was 298 items directly inside /etc.





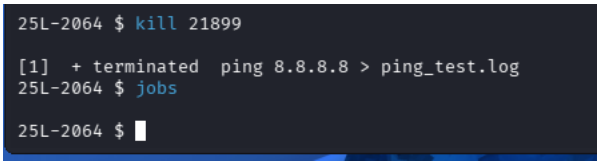
Explanation:

I created the user secanalyst with a home folder and Bash as the shell, then set its password. Then I created the group soc_team and added the user to it with usermod -aG. Then I made incident_report.txt and used chmod 760, so the owner has full access, the group can read and write, and others have no access. Then I used chgrp to change the file's group to soc_team, and ls -l shows the permissions -rwxrw---- and the group soc_team.



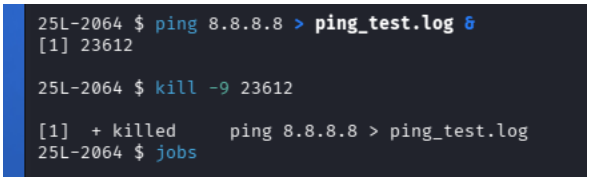
Explanation:

I started ping 8.8.8.8 > ping_test.log &. The & runs it in the background, so I could still use the terminal. jobs -l and ps aux | grep ping show the ping is running with PID 21899.



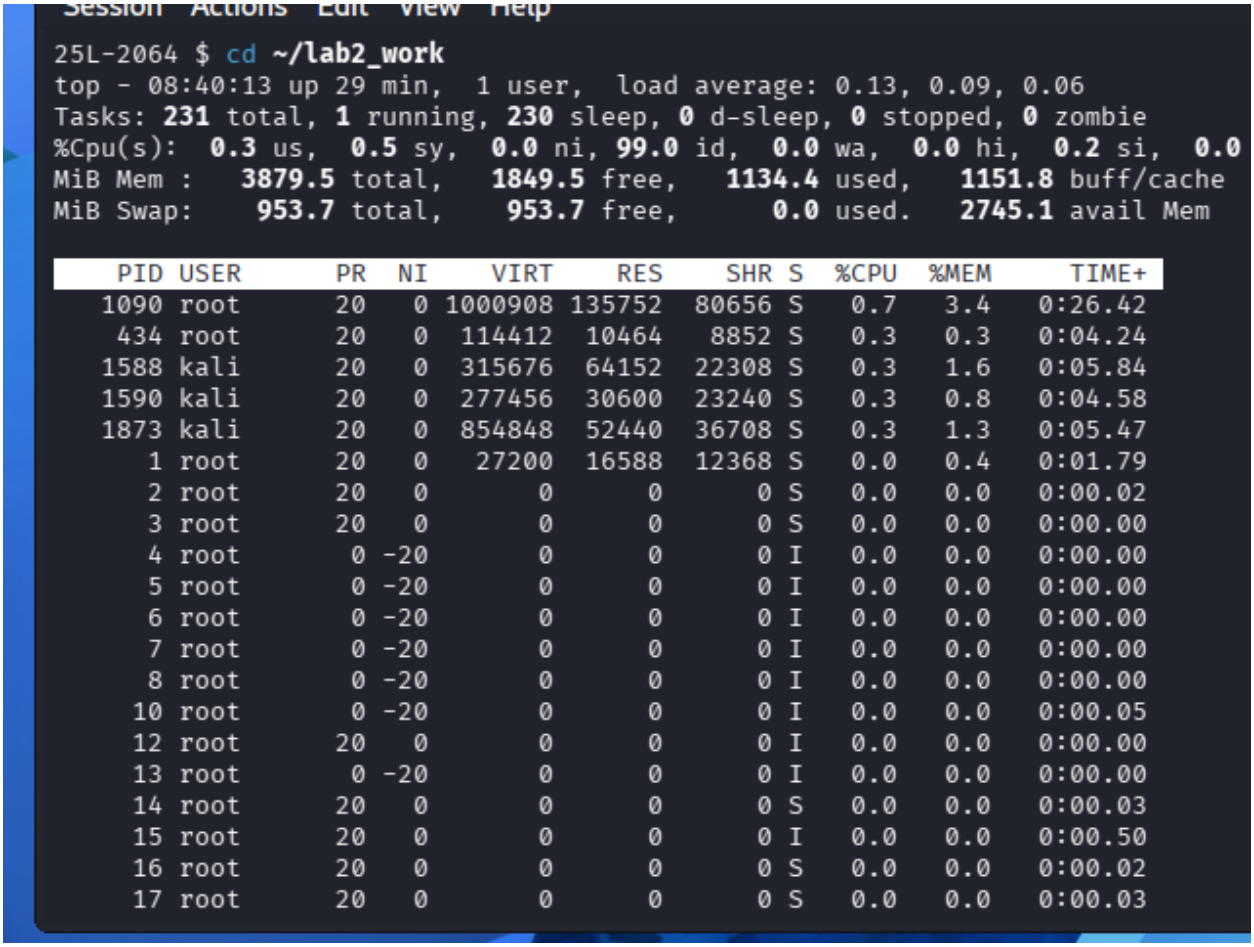
Explanation:

I stopped the ping with kill 21899. This sends SIGTERM, a polite request to stop. The terminal shows “terminated”, and jobs shows nothing left running.



Explanation:

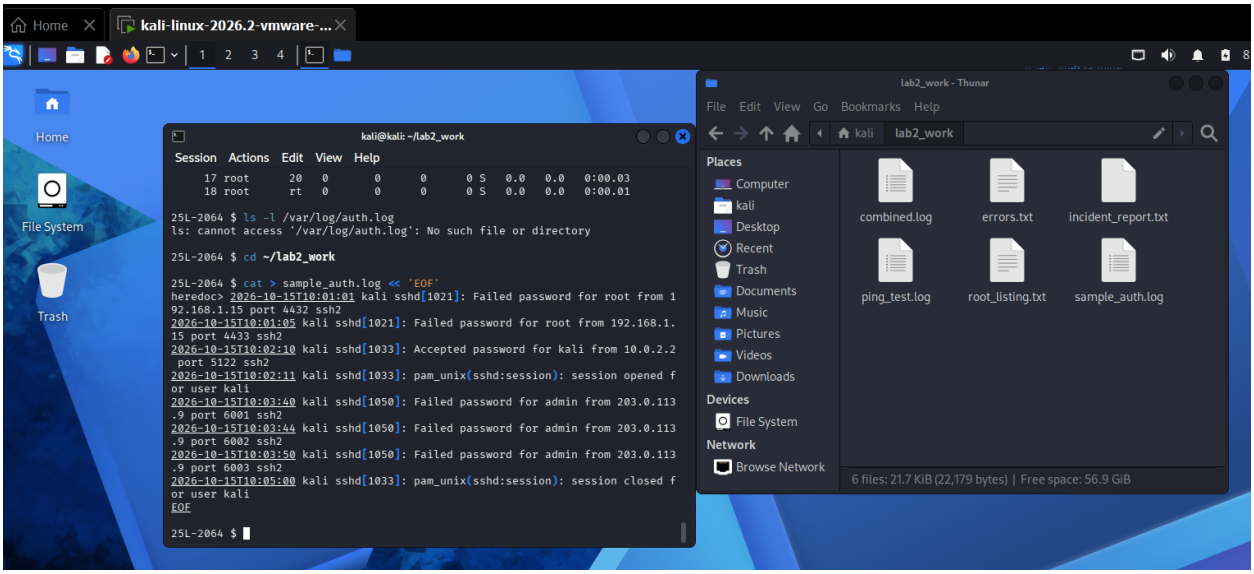
I started the ping again, which got a new PID (23612), and used kill -9 23612. This sends SIGKILL, which stops the process immediately and cannot be ignored. The terminal shows “killed”.



Explanation:

I ran top to see the live list of processes, sorted by CPU use. The processes using the most CPU were 1090 ,434 and 1588. After taking the screenshot I pressed q to quit.

1. **PID 1090** (0.7% CPU) - User: root
2. **PID 434** (0.3% CPU) - User: root
3. **PID 1588** (0.3% CPU) - User: kali
4. **PID 1590** (0.3% CPU) - User: kali
5. **PID 1873** (0.3% CPU) - User: kali



Explanation:

I checked for /var/log/auth.log and it doesn't exist on this Kali install, because the system logs through journald. So I created my own sample log file, sample_auth.log, with fake login entries and IP addresses to practice on.

```
25L-2064 $ ls -l sample_auth.log
-rw-rw-r-- 1 kali kali 748 Oct 16 08:43 sample_auth.log

25L-2064 $ cat sample_auth.log
2026-10-15T10:01:01 kali sshd[1021]: Failed password for root from 192.168.1.
15 port 4432 ssh2
2026-10-15T10:01:05 kali sshd[1021]: Failed password for root from 192.168.1.
15 port 4433 ssh2
2026-10-15T10:02:10 kali sshd[1033]: Accepted password for kali from 10.0.2.2
 port 5122 ssh2
2026-10-15T10:02:11 kali sshd[1033]: pam_unix(sshd:session): session opened f
or user kali
2026-10-15T10:03:40 kali sshd[1050]: Failed password for admin from 203.0.113
.9 port 6001 ssh2
2026-10-15T10:03:44 kali sshd[1050]: Failed password for admin from 203.0.113
.9 port 6002 ssh2
2026-10-15T10:03:50 kali sshd[1050]: Failed password for admin from 203.0.113
.9 port 6003 ssh2
2026-10-15T10:05:00 kali sshd[1033]: pam_unix(sshd:session): session closed f
or user kali
```

Explanation:

I used ls -l and cat to confirm the file was created and contains the log lines.

```
25L-2064 $ grep -c "session" sample_auth.log
2
```

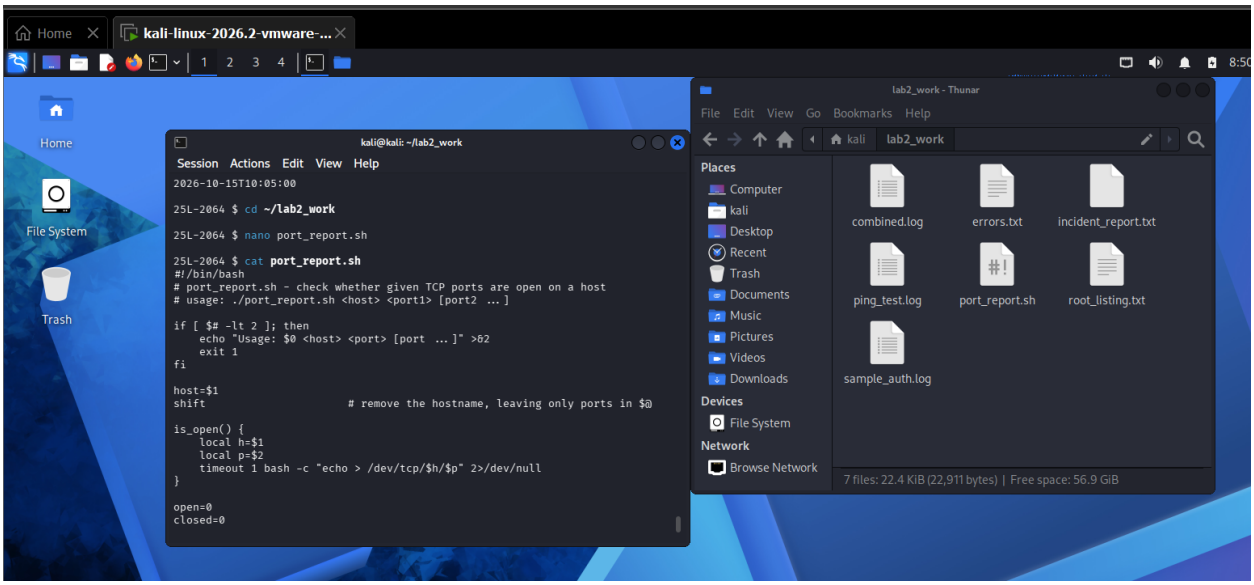
```
25L-2064 $ grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" sample_auth.log
192.168.1.15
192.168.1.15
10.0.2.2
203.0.113.9
203.0.113.9
203.0.113.9

25L-2064 $ grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" sample_auth.log | sort | un
iq -c | sort -rn
      3 203.0.113.9
      2 192.168.1.15
      1 10.0.2.2

25L-2064 $ awk '{print $1}' sample_auth.log
2026-10-15T10:01:01
2026-10-15T10:01:05
2026-10-15T10:02:10
2026-10-15T10:02:11
2026-10-15T10:03:40
2026-10-15T10:03:44
2026-10-15T10:03:50
2026-10-15T10:05:00
```

Explanation:

grep -c "session" counted the lines with the word “session” (2 lines). grep -oE with an IPv4 pattern pulled out every IP address. Then sort | uniq -c | sort -rn counted and ranked them: 203.0.113.9 appeared 3 times, 192.168.1.15 twice and 10.0.2.2 once. Finally, awk '{print $1}' printed only the first column, the timestamp.



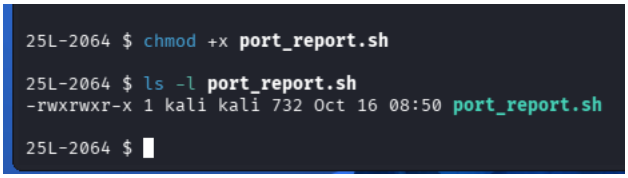
```
open=0
closed=0

for port in "$@"; do
    if is_open "$host" "$port"; then
        echo "[OPEN]   $host:$port"
        open=$((open + 1))
    else
        echo "[CLOSED] $host:$port"
        closed=$((closed + 1))
    fi
done

echo "————————————————————————————————"
echo "Summary for $host: $open open, $closed closed"
exit 0
```

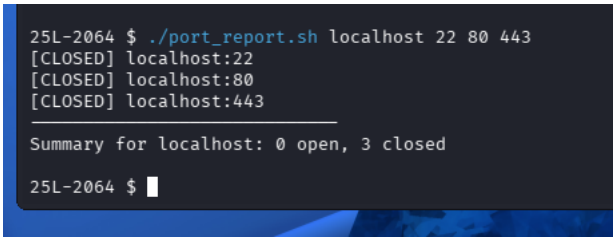
Explanation:

I created port_report.sh with nano. The script checks the arguments, saves the host, uses shift, and has a function is_open() that tests a port using /dev/tcp. I displayed it with cat to show the code



Explanation:

I used chmod +x port_report.sh so the script can run. ls -l shows the x permission. The script loops over every port, calls is_open, counts open and closed ports, prints a summary and ends with exit 0.



Explanation:

I ran ./port_report.sh localhost 22 80 443. All three ports were closed because no service was listening, and the summary shows 0 open and 3 closed.

```
25L-2064 $ sudo systemctl start ssh
[sudo] password for kali:

25L-2064 $ ./port_report.sh localhost 22 80 443
[OPEN]   localhost:22
[CLOSED] localhost:80
[CLOSED] localhost:443
--------------------------------
Summary for localhost: 1 open, 2 closed
```

Explanation:

I started the SSH service with sudo systemctl start ssh and ran the script again. This time port 22 shows OPEN, and the summary shows 1 open and 2 closed, which proves the script detects open ports correctly.

```
25L-2064 $ sudo systemctl stop ssh

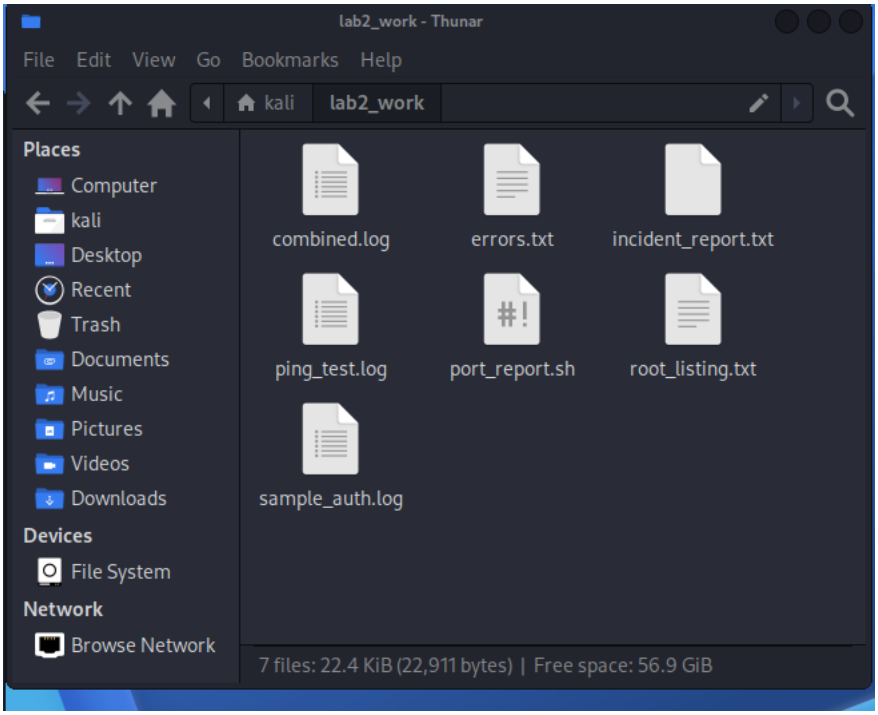
25L-2064 $ ./port_report.sh
Usage: ./port_report.sh <host> <port> [port ...]

25L-2064 $ echo $?
1

25L-2064 $
```

Explanation:

I stopped SSH again and ran the script with no arguments. It printed the usage message, and echo $? showed exit code 1, meaning failure. This shows the script handles wrong input properly.



This shows all the files I created in lab2_work.

Conceptual Questions (Q26-33)

26. Why are stdout and stderr separate?
stdout carries the normal results of a program and stderr carries error messages. Keeping them apart helps me to save the results to a file and I can still see errors on screen, or I can hide the errors and keep only the results. It is important for scripts and scanners, where an error line in the results file would spoil the data.

27. command > file 2>&1 vs command 2>&1 > file
The shell reads redirections from left to right. In > file 2>&1, stdout goes to the file first, and then stderr is pointed to the same place as stdout, so both end up in the file. In 2>&1 > file, stderr is pointed to where stdout is now, which is still the screen, and only then does stdout move to the file. So, the errors still show on screen and only the normal output goes to the file.

28. Permission 777 vs 750
777 gives read, write and execute to everyone on the system. Any user or program could change or replace the file, for example to hide malicious code in a script that root runs later, which can lead to privilege escalation. 750 gives the owner full access, the group read and execute, and others nothing, so only the people who need the file can use it. That follows the principle of least privilege.

29. Why /etc/shadow is root-only but /etc/passwd is readable by all
Many normal programs need /etc/passwd to match user ID numbers to usernames, and it does not store passwords. /etc/shadow holds the hashed passwords. If anyone could read it, an attacker could copy the hashes and crack them offline, so only root may read it.

30. SIGTERM vs SIGKILL
SIGTERM (15) politely asks a process to stop, and the process can save its work and close files first. SIGKILL (9) forces the process to stop immediately, and it cannot be caught or ignored, so the process gets no chance to clean up. An admin should try SIGTERM first and use SIGKILL only if the process is frozen or ignores SIGTERM.

31. Example where regex saves time
After a suspected brute-force attack, a log file may have thousands of lines. With one command like grep -oE for the IP pattern, then sort | uniq -c | sort -rn, I can pull out all attacker IP addresses and see which ones tried most often, in seconds. Reading the file by eye could take hours and I could miss lines.

32. Why use local in functions
local makes a variable exist only inside the function. Without it, the variable is global and could overwrite another variable with the same name in the main script, causing bugs that are hard to find. Using local keeps functions safe and independent.

33. Non-zero exit codes, && and ||
Every command ends with an exit code. 0 means success, and any non-zero number means something failed. && runs the next command only if the previous one returned 0, and || runs the next command only if the previous one returned non-zero. For example, mkdir test && cd test only changes into the folder if it was created. Scripts use this to stop or take another action when something goes wrong, like my script exiting with code 1 when no arguments are given.

port_report.sh source code text:

```
#!/bin/bash

# port_report.sh - check whether given TCP ports are open on a host

# usage: ./port_report.sh <host> <port1> [port2 ...]


if [ $# -lt 2 ]; then

  echo "Usage: $0 <host> <port> [port ...]" >&2

  exit 1

fi


host=$1

shift                # remove the hostname, leaving only ports in $@


is_open() {

  local h=$1

  local p=$2

  timeout 1 bash -c "echo > /dev/tcp/$h/$p" 2>/dev/null
```

```
}

open=0
closed=0

for port in "$@"; do
  if is_open "$host" "$port"; then
    echo "[OPEN]   $host:$port"
    open=$((open + 1))
  else
    echo "[CLOSED] $host:$port"
    closed=$((closed + 1))
  fi
done

echo "------------------------------"
echo "Summary for $host: $open open, $closed closed"
exit 0
```

15.1 Self-Check Skills Checklist

☐ I can explain the difference between stdout and stderr, and why keeping them separate is useful. 10/10

☐ I can redirect output to a new file, append to an existing file, and merge stderr into stdout correctly. 9/10

I can explain the difference between a pipe (|) and the && / || / ; control operators. 10/10

☐ I can read a permission string like -rwxr-xr-- and convert it to and from octal notation (e.g. 754). 10/10

☐ I can create a user, create a group, and assign a user to a group. 10/10

☐ I can find a running process by name and terminate it using an appropriate signal. 10/10

☐ I can write a grep command using at least three different regex metacharacters. 10/10

☐ I can explain what sed and awk are each best suited for, and how they differ from grep. 10/10

☐ I can look up an unfamiliar command's usage using man or --help without searching the internet. 10/10

☐ I can write a Bash script containing a variable, a conditional, a loop, and a function, and can explain what $1, $@, $#, and $? each mean. 8/10